

UML Class Diagram-BSSE1610

Factory Method · Abstract Factory · Builder · Dependency Injection

Github: [Creational Pattern](#)

Part A: Factory Method (package Exams)

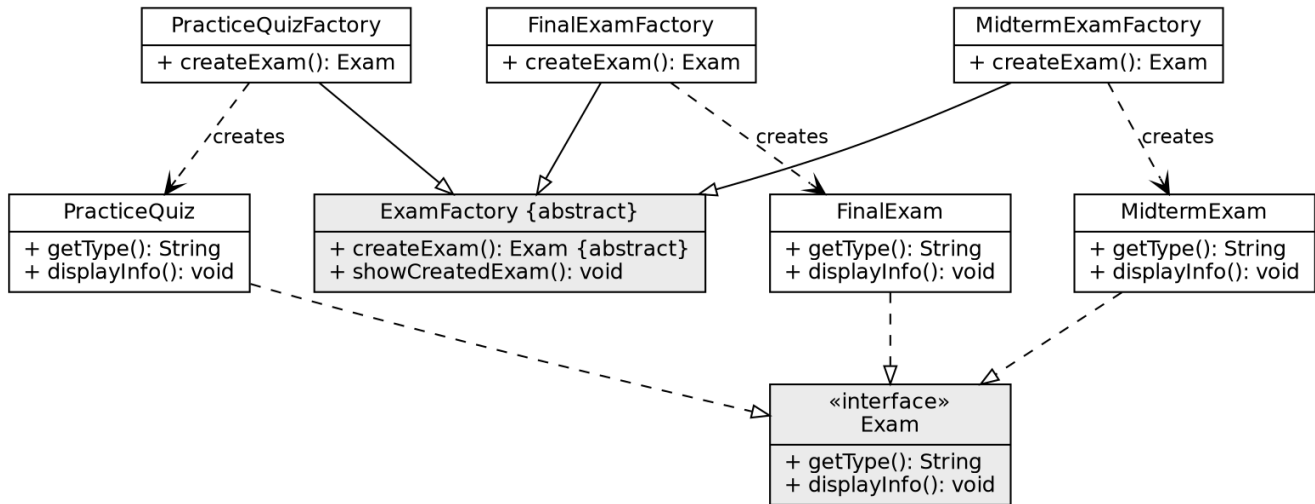
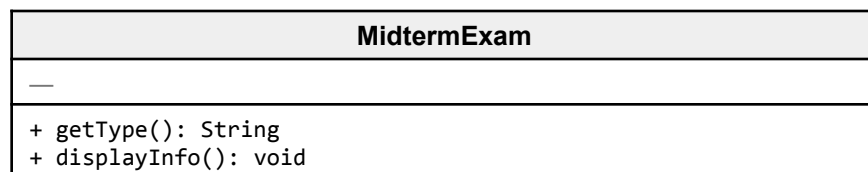
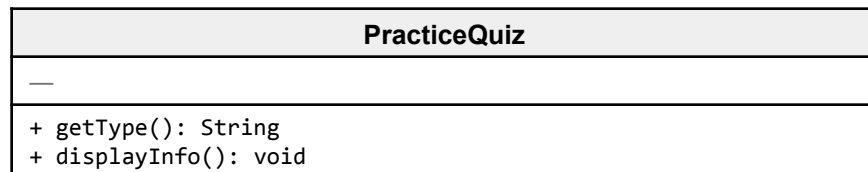
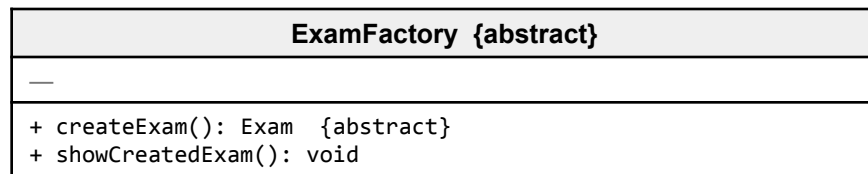
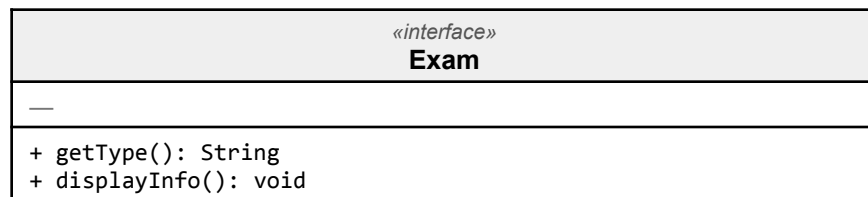


Figure A: Factory method hierarchy.

Class boxes



FinalExam
—
+ getType(): String + displayInfo(): void

PracticeQuizFactory
—
+ createExam(): Exam

MidtermExamFactory
—
+ createExam(): Exam

FinalExamFactory
—
+ createExam(): Exam

Part B (1 of 2): Abstract Factory: MCQ & True/False families

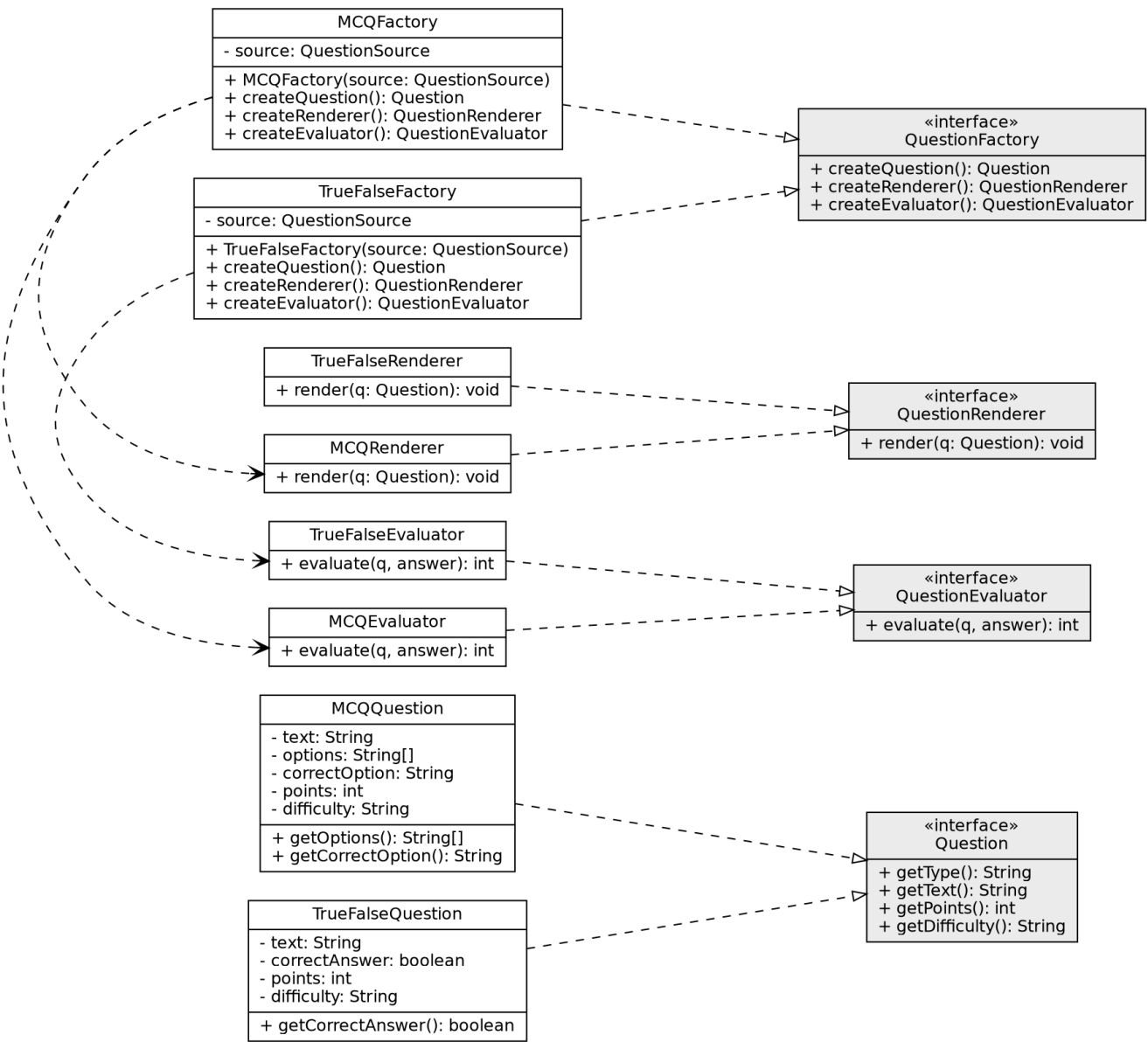
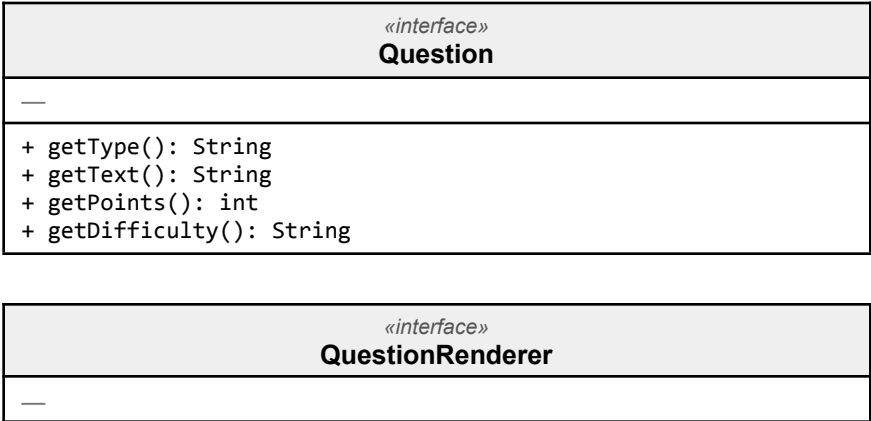


Figure B-1: MCQ and True/False families realizing the four product interfaces.

Class boxes, product interfaces



+ render(q: Question): void

«interface»

QuestionEvaluator

—

+ evaluate(q: Question, answer: String): int
--

«interface»

QuestionFactory

—

+ createQuestion(): Question + createRenderer(): QuestionRenderer + createEvaluator(): QuestionEvaluator
--

Class boxes — MCQ & True/False

MCQQuestion

- text: String - options: String[] - correctOption: String - points: int - difficulty: String

+ getType()/getText()/getPoints()/getDifficulty() + getOptions(): String[] + getCorrectOption(): String

MCQRenderer

—

+ render(q: Question): void

MCQEvaluator

—

+ evaluate(q: Question, answer: String): int
--

MCQFactory

- source: QuestionSource

+ MCQFactory(source: QuestionSource) + createQuestion(): Question + createRenderer(): QuestionRenderer + createEvaluator(): QuestionEvaluator
--

TrueFalseQuestion

- text: String - correctAnswer: boolean - points: int - difficulty: String

+ getType()/getText()/getPoints()/getDifficulty() + getCorrectAnswer(): boolean
--

TrueFalseRenderer

—

+ render(q: Question): void

TrueFalseEvaluator

—

+ evaluate(q: Question, answer: String): int
--

TrueFalseFactory

- source: QuestionSource

+ TrueFalseFactory(source: QuestionSource) + createQuestion(): Question + createRenderer(): QuestionRenderer + createEvaluator(): QuestionEvaluator
--

Part B (2 of 2): Abstract Factory: Essay & Programming families

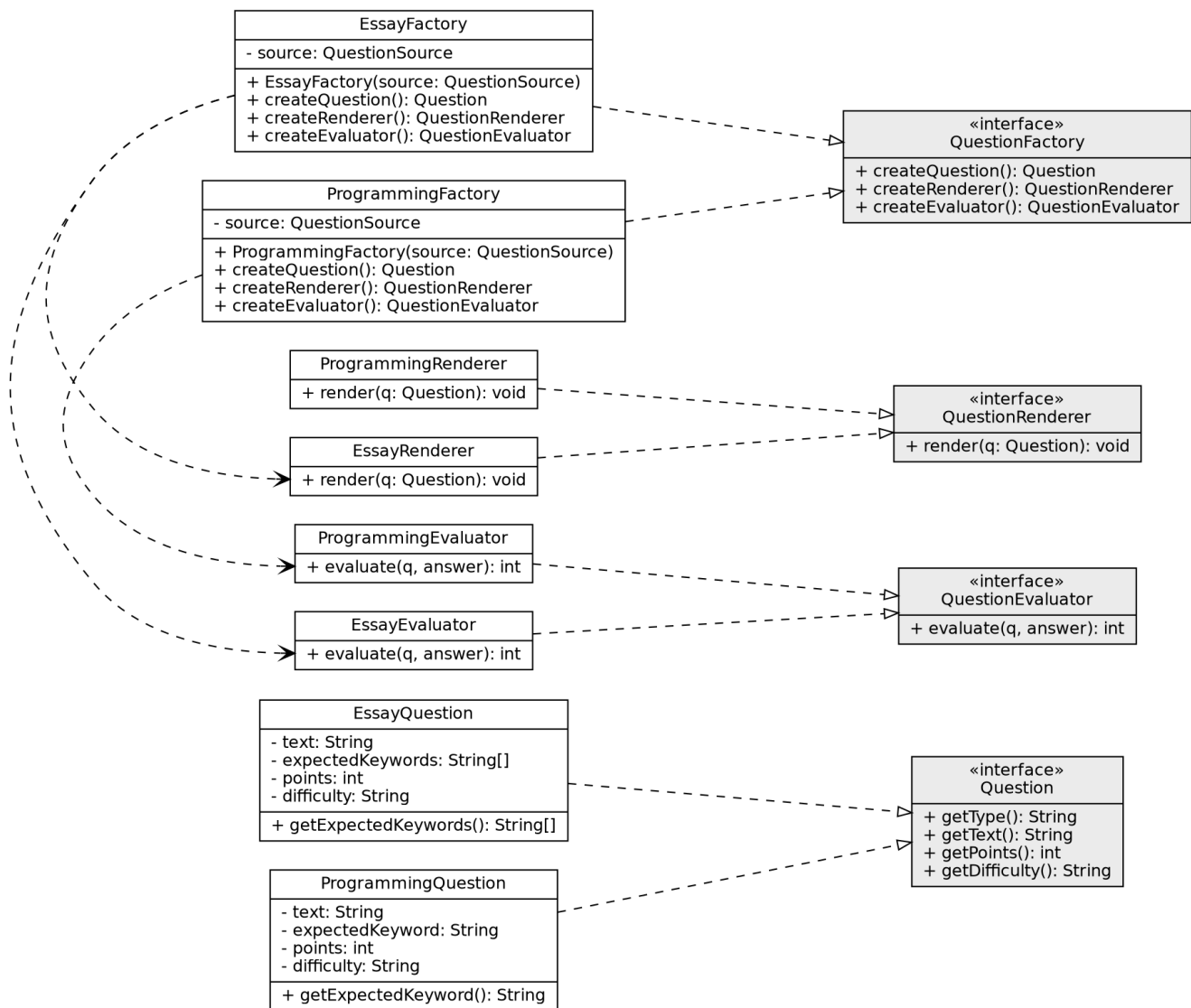
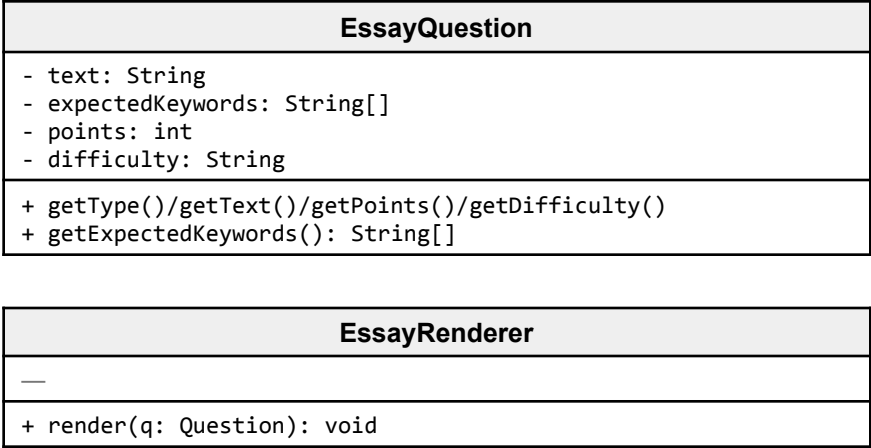


Figure B-2: Essay and Programming families realizing the four product interfaces.

Class boxes — Essay & Programming



EssayEvaluator
—
+ evaluate(q: Question, answer: String): int

EssayFactory
- source: QuestionSource
+ EssayFactory(source: QuestionSource)
+ createQuestion(): Question
+ createRenderer(): QuestionRenderer
+ createEvaluator(): QuestionEvaluator

ProgrammingQuestion
- text: String
- expectedKeyword: String
- points: int
- difficulty: String
+ getType()/getText()/getPoints()/getDifficulty()
+ getExpectedKeyword(): String

ProgrammingRenderer
—
+ render(q: Question): void

ProgrammingEvaluator
—
+ evaluate(q: Question, answer: String): int

ProgrammingFactory
- source: QuestionSource
+ ProgrammingFactory(source: QuestionSource)
+ createQuestion(): Question
+ createRenderer(): QuestionRenderer
+ createEvaluator(): QuestionEvaluator

Part C: Builder (package Exams)

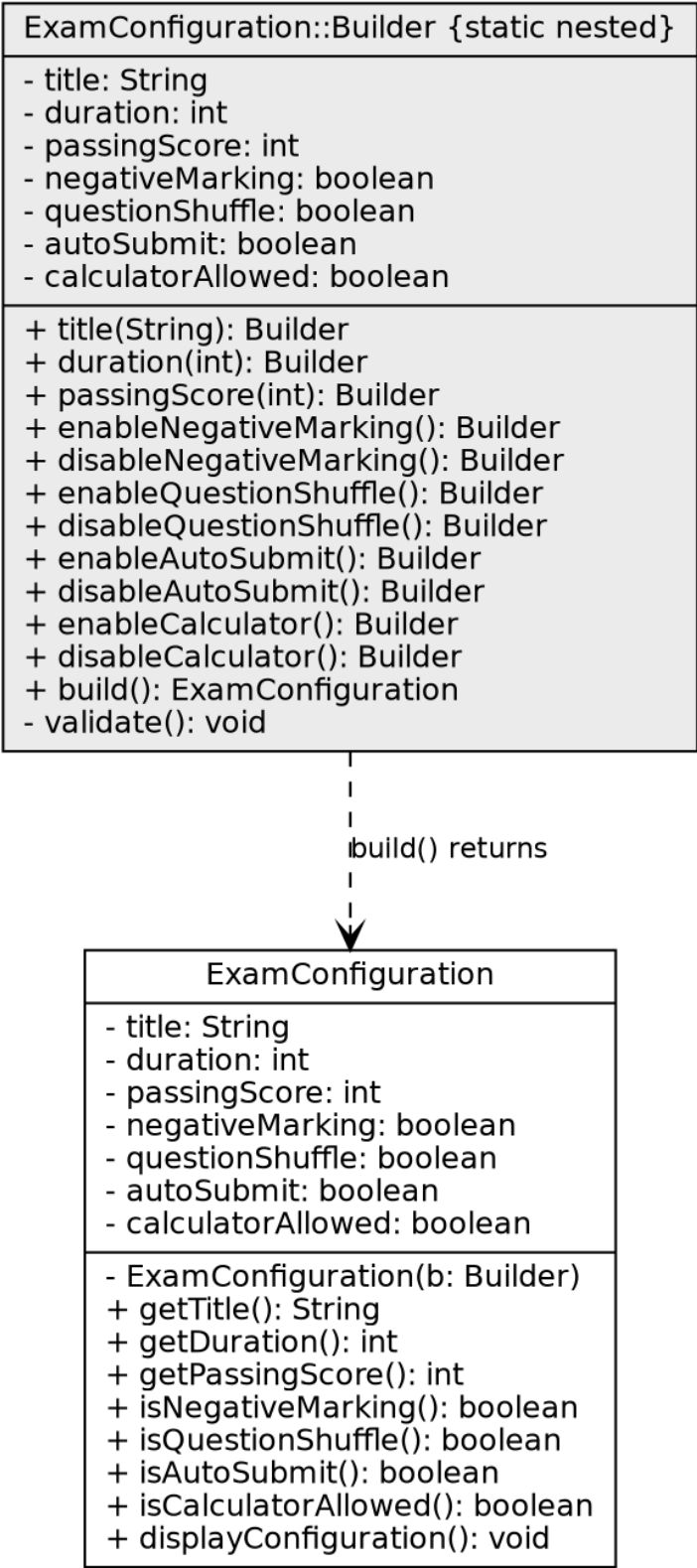


Figure C: Builder and the immutable ExamConfiguration it constructs.

Class boxes



- title: String - duration: int - passingScore: int - negativeMarking: boolean - questionShuffle: boolean - autoSubmit: boolean - calculatorAllowed: boolean
- ExamConfiguration(b: Builder) + getTitle(): String + getDuration(): int + getPassingScore(): int + isNegativeMarking(): boolean + isQuestionShuffle(): boolean + isAutoSubmit(): boolean + isCalculatorAllowed(): boolean + displayConfiguration(): void

ExamConfiguration::Builder {static nested}

- title: String - duration: int - passingScore: int - negativeMarking: boolean - questionShuffle: boolean - autoSubmit: boolean - calculatorAllowed: boolean
+ title(String): Builder + duration(int): Builder + passingScore(int): Builder + enableNegativeMarking(): Builder + disableNegativeMarking(): Builder + enableQuestionShuffle(): Builder + disableQuestionShuffle(): Builder + enableAutoSubmit(): Builder + disableAutoSubmit(): Builder + enableCalculator(): Builder + disableCalculator(): Builder + build(): ExamConfiguration - validate(): void

Part D: Question Sourcing & Dependency Injection (package Questions.QSource)

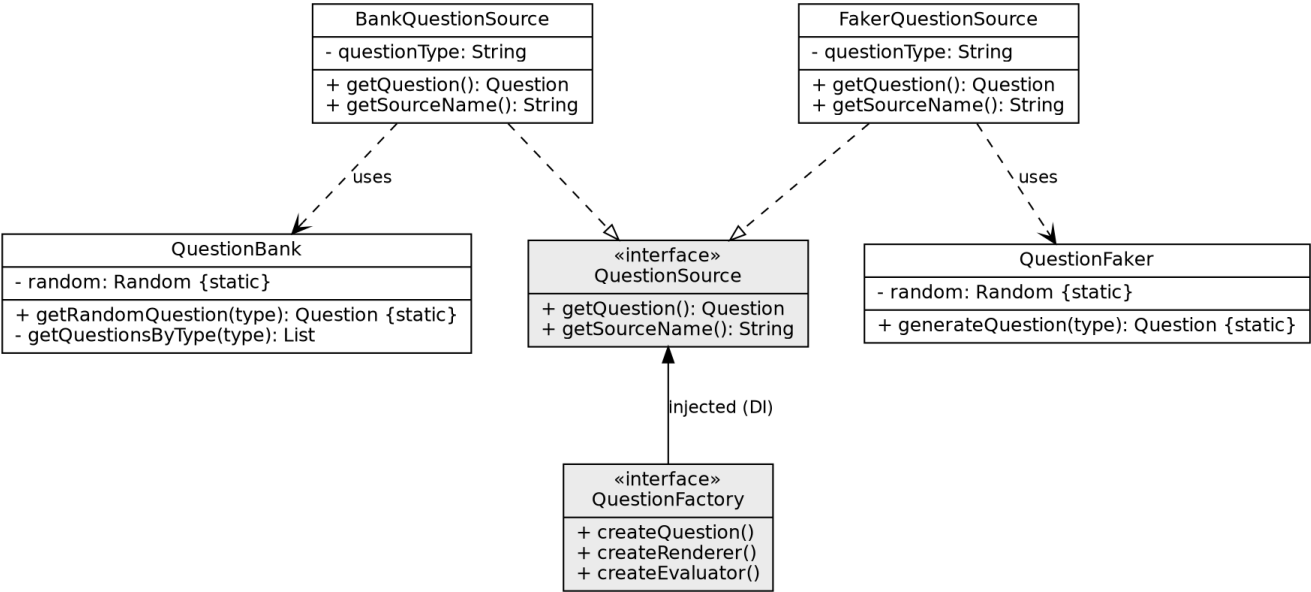


Figure D : QuestionSource abstraction injected into the question factories.

Class boxes

«interface» QuestionSource
—
+ getQuestion(): Question + getSourceName(): String

BankQuestionSource
- questionType: String
+ BankQuestionSource(questionType: String) + getQuestion(): Question + getSourceName(): String

FakerQuestionSource
- questionType: String
+ FakerQuestionSource(questionType: String) + getQuestion(): Question + getSourceName(): String

QuestionBank
- random: Random {static}
+ getRandomQuestion(type: String): Question {static} - getQuestionsByType(type: String): List<Question> {static}

QuestionFaker
- random: Random {static}
+ generateQuestion(type: String): Question {static}

Main
—
+ main(args: String[]): void {static} - printDashboard(exam, config, sourceName, questions): void {static}